The Development Road

An Nguyen

CS110 · Action 11B · Personal record
Exported 11/5/2026, 4:08:13 PM PST

Recorded work, not a grade or submission receipt. Missing entries remain marked. Review personal details before sharing.

Personal GitHub: https://github.com/anguyen0344/CS-110

Before the next challenge.

My name

An Nguyen

A question I want to resolve

How can I turn my moving cube from Action 11A into a playable scene where the player controls an object and has a clear goal?

Recall. Question. Recalibrate.

What have I completed?

My previous work:
https://github.com/anguyen0344/CS-110/tree/main/Week11A

I created a Unity cube with separate movement, rotation, and color scripts. I tested Speed 0, 1, and 2 and changed the startup color to red. I also made a boundary-stop experiment and compared two prefab instances.

My explanation of classes, objects, data and actions

A class defines data and behavior. Each component attached to a cube is an instance of its script class.

In TranslateObject, the speed field stores the movement rate, and the Speed property provides get/set access.
Unity calls Update() every frame. The method uses transform.Translate() with Speed and Time.deltaTime to move the cube.

Visual Studio is where I edit the C# code.
Unity runs the scene and displays the result.

My questions, answers and remaining uncertainty

Question: How can I turn automatic cube movement into player-controlled movement with a clear goal?

What I already understand: Separate components control movement, rotation, and color.

Next investigation: Learn how to read player input and detect when the player reaches a goal.
I will use the lesson, official Unity documentation, and instructor help when needed.

My personal GitHub repository URL

https://github.com/anguyen0344/CS-110

Readiness check

I reopened my 11A Unity project and tested the scene. My saved work includes the base scene, extension scene, scripts, and prefab. My GitHub folder contains the source, notes, receipt, and evidence. The repository link opens.

Replace Chapters with Pipelines

Attention First, Chapter 3, pp. 37–41 (132-page edition).

What I understand from this chapter

A pipeline organizes small, connected actions toward a visible outcome. Each node should show where I am, why the step matters, and what it prepares me to do next (pp. 37–38).

The chapter describes three core pipelines:
Important Information helps learners understand the experience.
The Beginning prepares their tools, organization, and expectations.
Storytelling gives the work meaning and guides technical choices (pp. 38–39).

The visual hat focuses on how people perceive and use the result.
The logical hat focuses on data, structure, and behavior.
Both support the same story (p. 40).

My main takeaway is to define the outcome first and work backward to choose the steps needed to reach it (p. 41).

My opinion and reasons

I agree that small steps are more useful when they connect to a clear outcome. In 11A, I made a cube move, rotate, and change color. For my next scene, I want those features to support something the player can actually do.

I also think preparation matters. Checking my project and GitHub first can prevent confusion about which copy I am editing. I would keep checkpoints so I can pause and return to my work.

A decision I can apply

My proposed route:

1. Story and goal:
The player guides a cube to a target.
Ready when: The goal and success condition are clear.

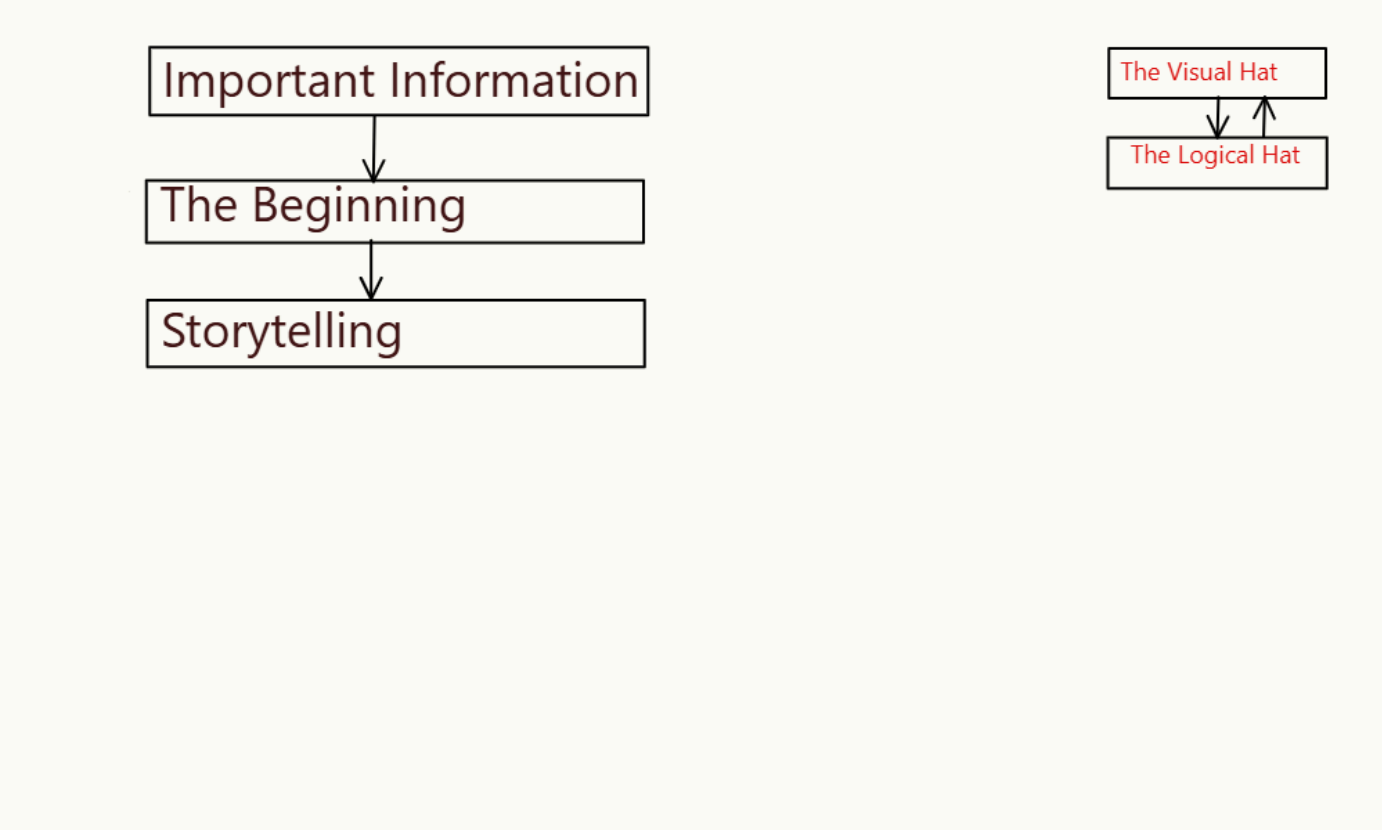
2. Controls and scene:
Create a player cube, a target, and keyboard movement.
Ready when: The scene objects and movement script are connected.

3. Tested player action:
The player moves the cube and reaches the target.
Ready when: The scene gives visible success feedback
and I have tested the controls.

A question or next investigation

How can I detect that the player has reached the target
and display clear success feedback?

Personal whiteboard · An Nguyen



An Nguyen: I created this diagram.
It connects a proposed player goal to the controls,
scene setup, and a testable result.
No external images were used.

Design for the Attention Economy

Attention First, Chapter 6, pp. 53–58 (132-page edition).

What I understand from this chapter

The chapter explains that attention depends partly on how an experience is designed. Unclear instructions, clutter, and unnecessary switching can make it harder to focus (pp. 53–55).

The author describes four attention time scales: 5 seconds for initial interest, 5 minutes for useful action, 5 hours for sustained engagement, and 5 years for identity (p. 54).

My takeaway is that more technology is not always better. A clear priority, fewer distractions, and visible progress can support learning (pp. 56–58).

My opinion and reasons

I agree that unnecessary switching can interrupt progress. During 11A, I moved between Unity, my code editor, GitHub, and the assignment page. Each tool had a purpose, but I sometimes needed to check which file or project I was using.

I think a clear work sequence would help more than simply adding another tool.

A decision I can apply

Attention leak: Switching between instructions, code, Unity, and GitHub without a clear next step.

Proposed change: Work on one small behavior at a time. Read the instruction, edit the script, test in Unity, and record the result before uploading to GitHub. Keep unrelated tabs closed.

Working agreement: Before switching tools, state the next action and save the current work.

Visible progress: Keep a short checklist showing the current task, test result, and next step.

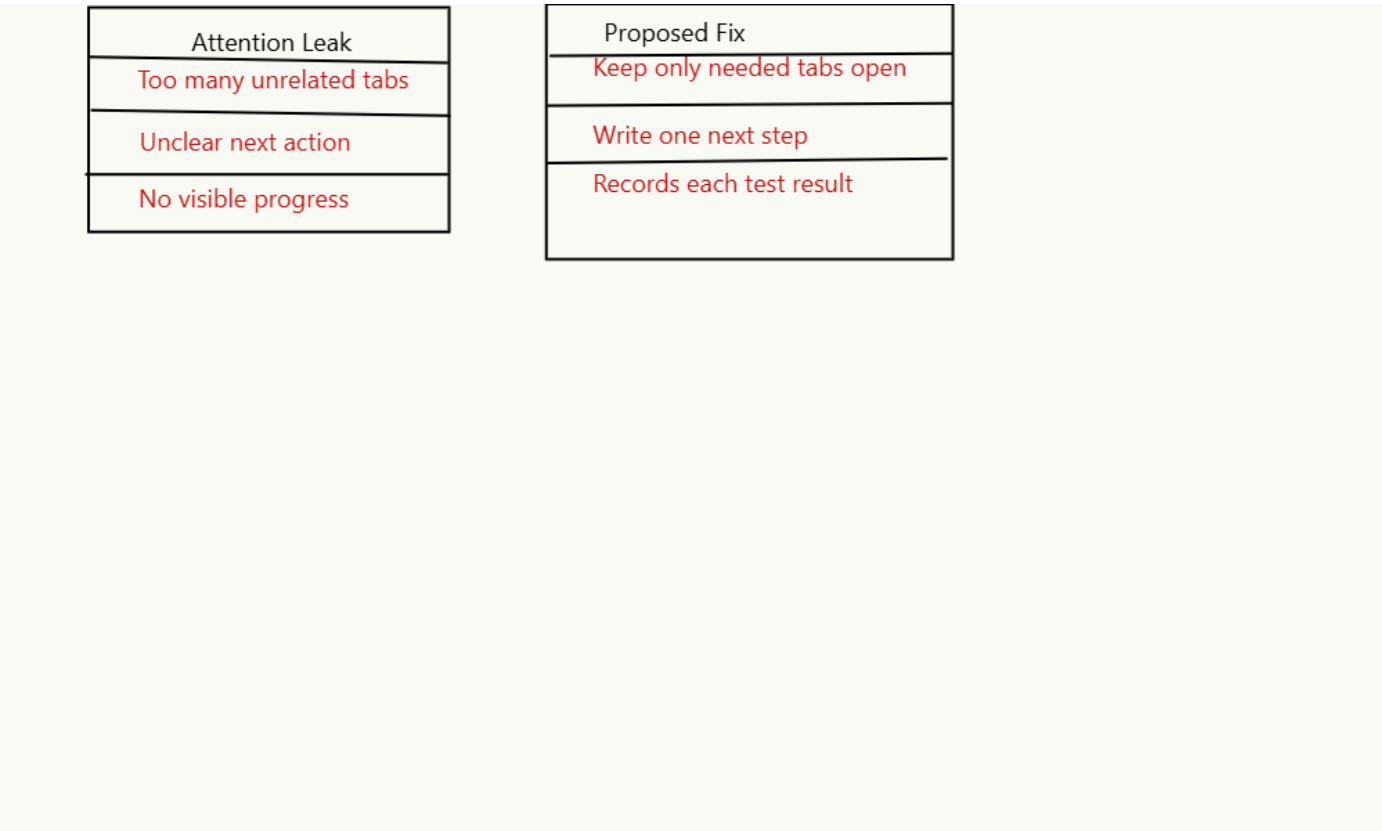
How I will check: Try this sequence during one work block and record whether I lose my place or reopen the wrong file less often.

A question or next investigation

Does working on one behavior at a time help me switch between Unity and my editor without losing my place?

I will test this during my next build session and record one thing that helped and one thing that still interrupted me.

Personal whiteboard · An Nguyen



No board explanation recorded.

Identity and the Creator Loop

Attention First, Chapter 7, pp. 61–69 (132-page edition).

What I understand from this chapter

The chapter describes identity as a pattern of repeated behavior, rather than a title or a confident statement (p. 61).

The canvas-and-paint metaphor suggests that visible behavior does not tell the whole story of a person. Growth involves changing patterns, not just improving appearances (pp. 62–65).

The creator loop connects practice, mistakes, reflection, and refinement. A successful result is a checkpoint, not the end of learning (pp. 67–68).

Principles help guide decisions when a checklist does not cover the situation. Reliable creators protect their standards and support their claims with completed work (pp. 68–69).

My opinion and reasons

I agree that responding to mistakes shows more than simply saying I am good at programming. In 11A, my boundary script did not stop the cube at first. I checked where the script was attached and moved it onto MyMarker.

That experience showed me the value of checking the setup and testing again. I still need practice, but I can build reliability through small, repeatable actions.

A decision I can apply

Past mistake: I attached StopAtBoundary to a separate object instead of the cube containing the movement component.

My response: I corrected the attachment and tested again. The cube then stopped at X = 3 while continuing to rotate.

Standard I will protect: Report actual results and keep working source and evidence together.

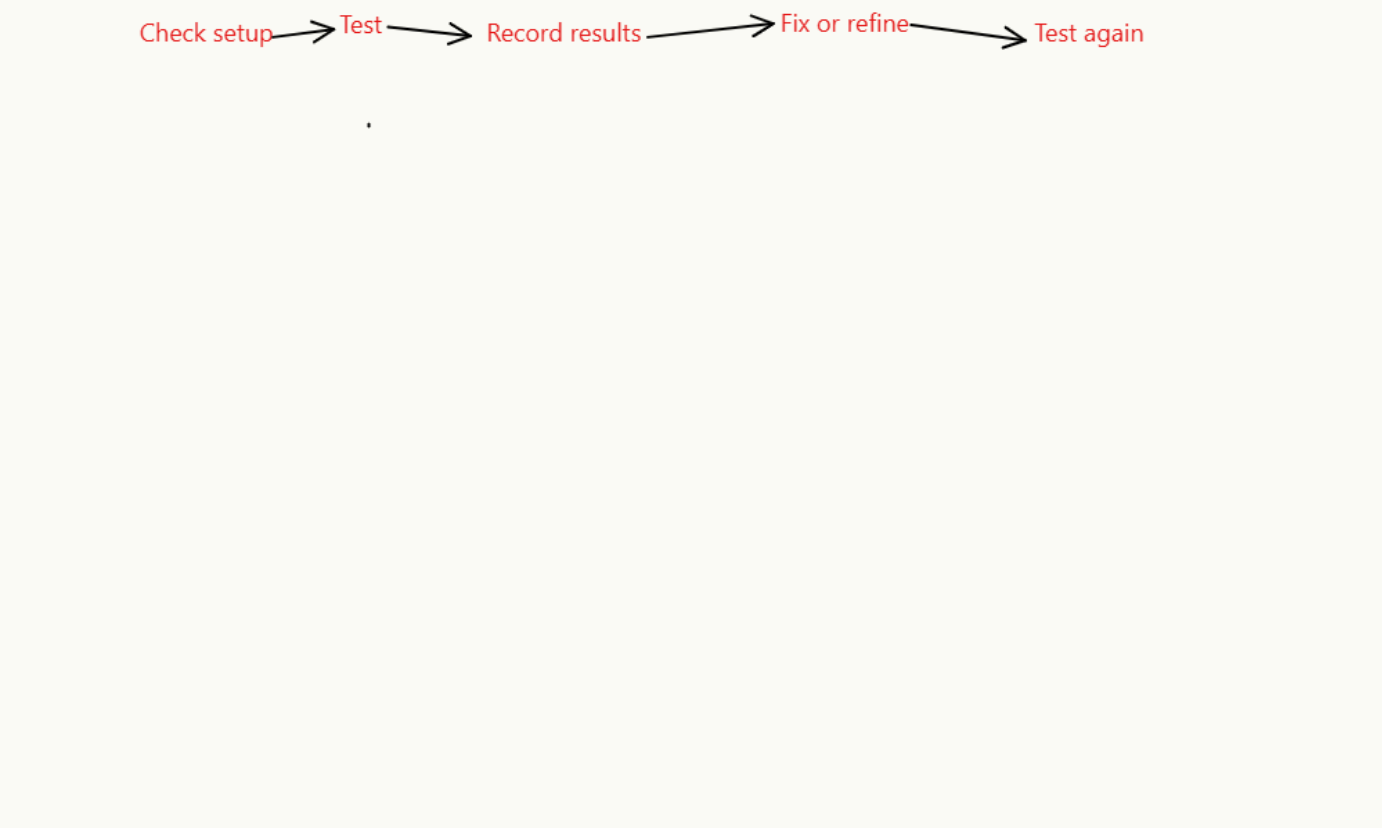
Repeatable action: Before each test, check the selected object, attached components, and starting values. After the test, record the result, save the work, and write the next step.

A question or next investigation

Can I follow the same check-test-record routine when I add player controls and a goal to my next scene?

I will try it during the next build and record whether it helps me find mistakes and return to unfinished work.

Personal whiteboard · An Nguyen



No board explanation recorded.

Designing Your Teams and Tools

Attention First, Chapter 8, pp. 71–74 (132-page edition).

What I understand from this chapter

Chapter 8 explains that teams and tools should be designed around the purpose of the work. A tool provides a useful function, while a teammate can contribute ideas and change how we think (pp. 71–72). The master focuses on the craft, while the chief coordinates the group; both roles matter (pp. 72–73). A useful workbook preserves history and helps the learner notice progress and return to the work (p. 74).

My opinion and reasons

I agree that tools should support the project's purpose. In my Unity work, the engine runs the scene, the code editor helps me change scripts, and GitHub stores the source and its history. My notes become more useful when they explain what I tested, what happened, and what I should do next.

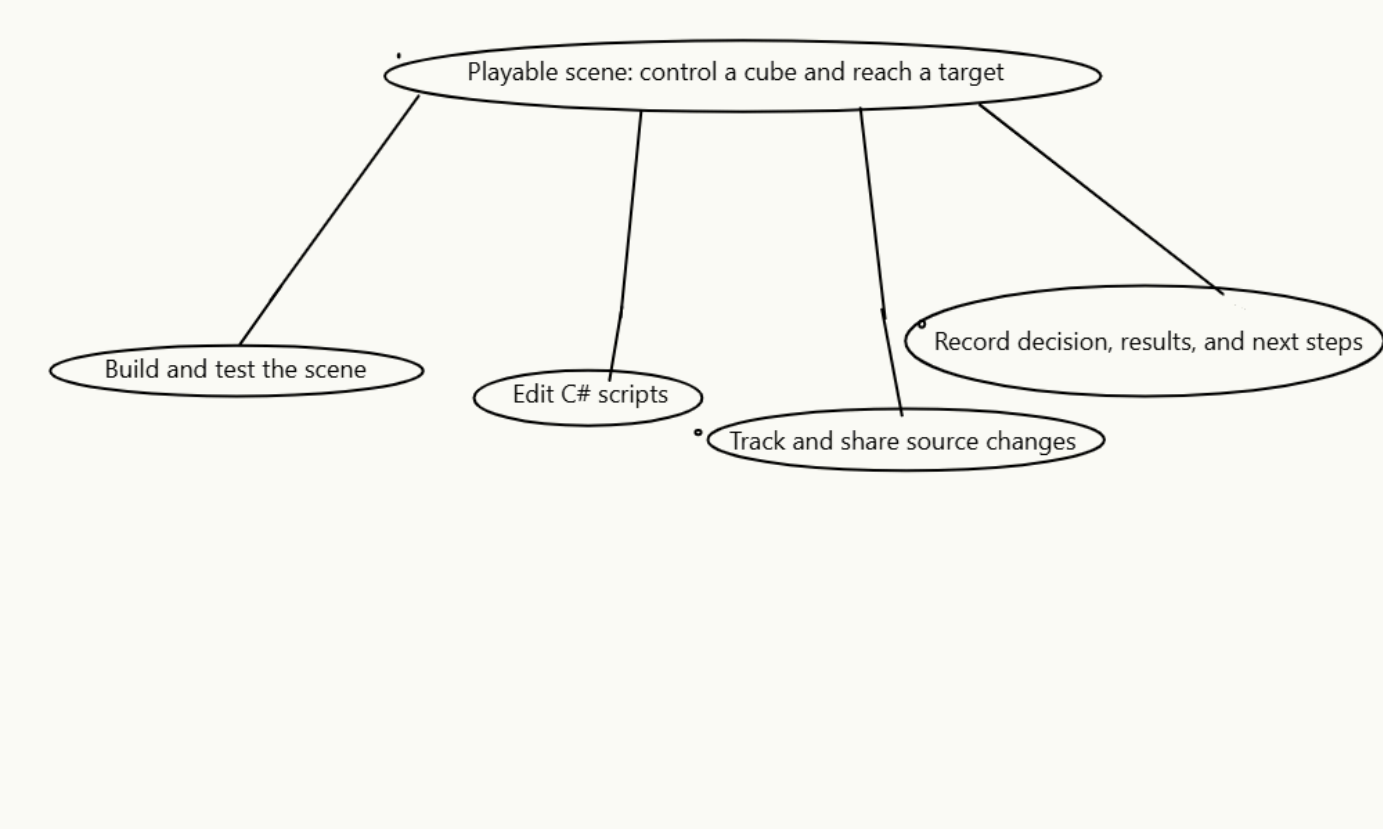
A decision I can apply

For Action 11B, I will give each tool a clear job: Unity for building and testing the scene, my code editor for editing C# scripts, GitHub for version history, and my README for decisions and test results. After each test, I will record the expected result, actual result, and next action. If I work with a team, I will confirm responsibilities with the members.

A question or next investigation

Can another person use my README to open the correct scene, understand the controls and goal, and repeat one test? I will check whether the instructions are clear and record anything that needs clarification.

Personal whiteboard · An Nguyen



No board explanation recorded.

The Human Operating System

Attention First, Chapter 11, pp. 93–103 (132-page edition).

What I understand from this chapter

Chapter 11 describes the human operating system as coordinating memory, attention, imagination, tools, and action (p. 93). It explains that intelligence alone is not wisdom; judgment helps us decide what matters and whether our actions create meaningful progress (pp. 94–95). Humans should guide tools through purpose, evaluation, and responsible decisions (pp. 96–97). The chapter also argues for balancing structure with freedom so that people can develop both discipline and creativity (pp. 101–103).

My opinion and reasons

I agree that using powerful tools does not automatically mean I understand my work. In Unity, a script may run, but I still need to explain its behavior and check whether it supports the player's goal. In Action 11A, testing Speed 0 and Speed 2 helped me connect the code to visible results.

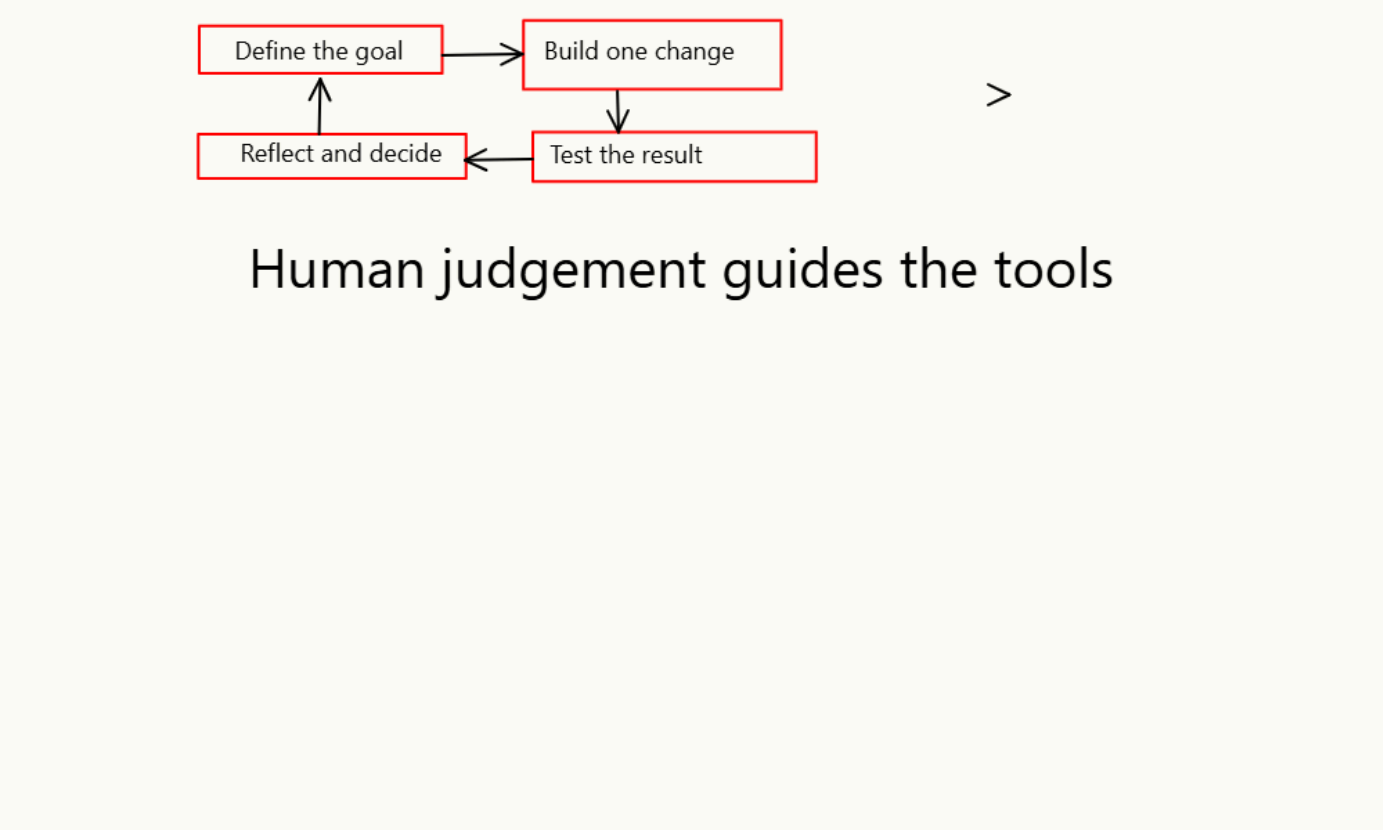
A decision I can apply

For Action 11B, I will define one player action and one clear goal before adding features. I will build and test one change at a time, compare the actual result with my prediction, and record the next step. If I use AI suggestions, I will explain and test the code before keeping it.

A question or next investigation

How can I tell whether a new feature improves the player's experience or only makes the scene more complicated? I will test whether the player can understand the controls, identify the goal, and recognize success.

Personal whiteboard · An Nguyen



No board explanation recorded.

Bring something to the summit.

The connection across the readings

The readings connect purpose, attention, practice, people, and tools. Chapter 3 helps me work backward from a visible outcome and divide the work into small steps. Chapter 6 reminds me to reduce distractions and make progress visible. Chapter 7 connects reliability to repeated habits such as testing and recording results. Chapter 8 helps me choose tools and clarify responsibilities. Chapter 11 reminds me to use my own judgment when coordinating these parts.

For Action 11B, my proposed mission is to turn my moving cube into a playable scene with player controls, a clear target, and visible success feedback.

Evidence I am ready to show

I can show my Action 11A Unity project, C# scripts, README, whiteboard, and screenshots in: https://github.com/anguyen0344/CS-110/tree/main/Week11A

My tests showed that the cube stayed still at Speed 0, moved faster at Speed 2, and turned red when I ran the scene again. These results demonstrate changes to movement and appearance.

I still need to learn how to connect player input to movement, detect reaching a target, and provide clear success feedback.

My board in words

My board connects my previous experience to a reading insight, a proposed mission, and supporting evidence.

Experience: I tested cube movement and color in Action 11A.
Reading insight: Work backward from a clear outcome and use small, repeatable tests.
Proposed mission: Let the player control a cube and reach a target.
Plan: Define the controls and goal, build one feature at a time, and test the result.
Evidence: Save scripts, scene changes, screenshots, and test notes in my personal repository.

Unity runs the scene, my code editor helps me edit scripts, and GitHub preserves the source history. I remain responsible for explaining and checking the results.

Personal whiteboard · An Nguyen



My board connects my previous experience to a reading insight, a proposed mission, and supporting evidence.

Experience: I tested cube movement and color in Action 11A.
Reading insight: Work backward from a clear outcome and use small, repeatable tests.
Proposed mission: Let the player control a cube and reach a target.
Plan: Define the controls and goal, build one feature at a time, and test the result.
Evidence: Save scripts, scene changes, screenshots, and test notes in my personal repository.

Unity runs the scene, my code editor helps me edit scripts, and GitHub preserves the source history. I remain responsible for explaining and checking the results.

Choose where you want to go.

My direction

Work solo

The mission I want to explore

I want to build a small Unity scene for a beginner player. The player controls a cube to reach a clearly marked target. The story moment is helping a lost cube reach its destination. Success should produce visible feedback so the player knows the goal is complete.

My pitch

My purpose is to turn my Action 11A cube experiment into a playable scene with controls, a goal, and success feedback. I bring working movement and color scripts, test screenshots, and notes from Action 11A. I still need to learn player input and target detection. My next action is to sketch the scene and define one movement test and one success test.

Conversations and decisions

I have chosen to work solo for this mission. I have not discussed a new team agreement yet. My current plan is to keep the scene small and build on my Action 11A work. I can ask the instructor or a classmate for help with input or target detection if I get stuck.

Agree on the work.

Team name / solo mission

Lost Cube: Reach the Target — Solo Mission

Agreed lead / my name if solo

An Nguyen — responsible for the mission's direction, scope, implementation, and testing.

Accepted members and contributions

An Nguyen — working solo. I will design the scene, implement player controls and target detection, test the results, and document my work. No other members are currently part of this mission.

How the literature strengthens this team

Chapter 3 informs my decision to work backward from the goal and divide the project into small, testable steps. Chapter 6 informs my decision to reduce distractions and make the controls and goal clear. Chapter 7 supports a repeatable testing habit. Chapter 8 helps me assign each tool a clear purpose while taking responsibility for both planning and building.

Communication and check-in plan

My source, notes, and evidence will live in my personal GitHub repository under Week11B. I will compare expected and actual results after each feature: player movement, target detection, and success feedback. Before finishing, I will reopen the saved scene and check the uploaded files. If I get stuck, I will show the instructor or a classmate the relevant code, error, and test result.

Team GitHub URL, or Working solo

Working solo

Make the Attention Accord.

Purpose and smallest playable result

The player helps a lost cube reach its destination. The smallest playable result is one scene where the player uses arrow keys to move the cube toward a clearly marked target. When the cube reaches the target, it turns green and displays “Goal reached!” The scene starts with a short instruction explaining the controls and goal.

My attention promise

Focus question: Can the player understand the controls, move the cube, and recognize success?

I will work on one feature at a time and record the expected and actual results after testing. I will capture the starting scene, movement, and success feedback, and keep these screenshots with my scripts and test notes. This evidence will help me explain the work and ask specific questions if I need help.

Time and dependency plan

First, I will prepare a separate Week11B project copy and sketch the scene (15 minutes). Next, I will implement and test keyboard movement (25 minutes). Target detection depends on working movement; I will add detection and success feedback next (25 minutes). Finally, I will test the complete scene, save evidence, update the README, and upload the work (25 minutes).

I will check progress after each feature. If blocked for more than 10 minutes, I will record the error, check the relevant setup, and ask for help. I will prioritize movement and one clear success condition before adding extra features.

Board explanation and contributors

My map connects character, goal, player action, condition, and visible consequence.

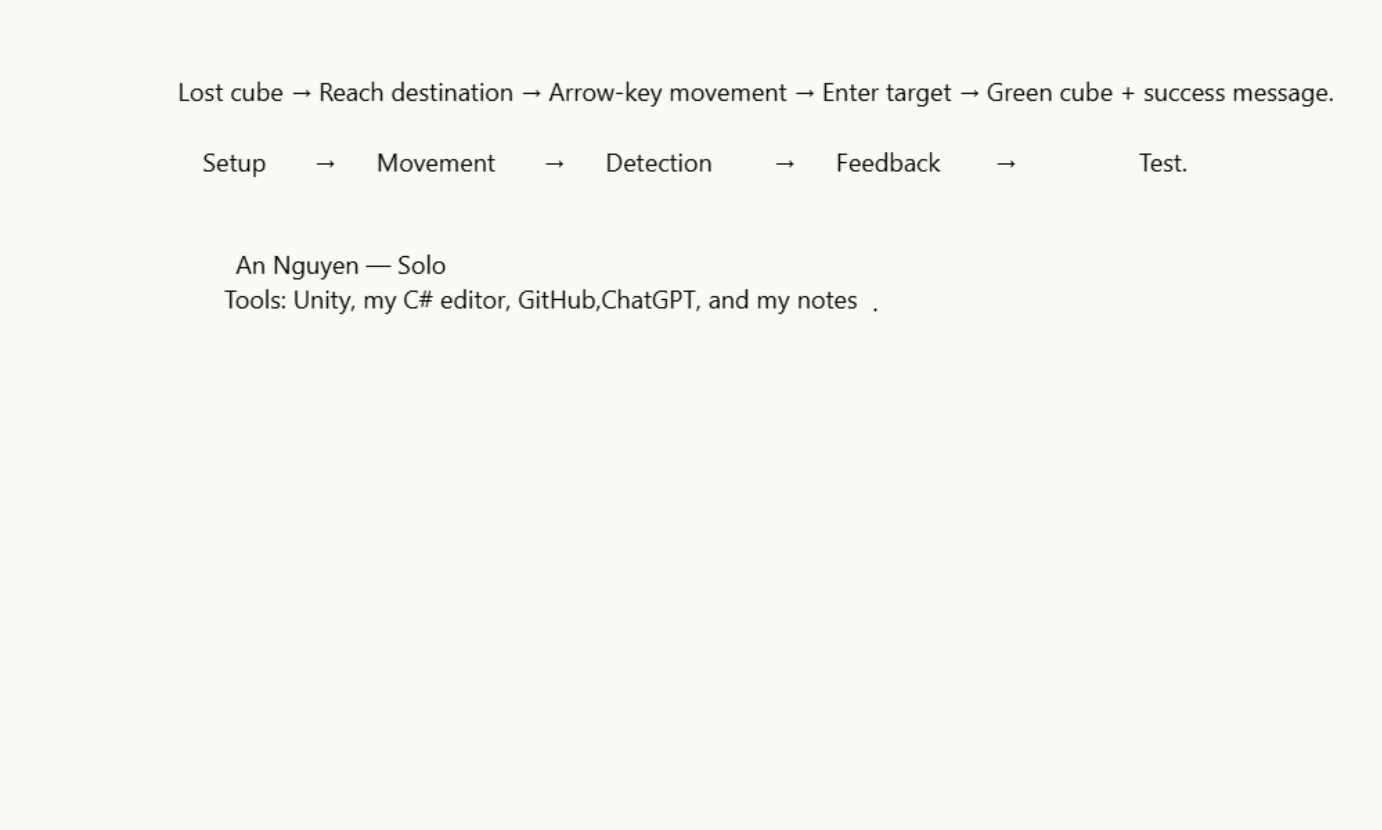
Character: a lost cube.
Goal: reach the marked destination.
Player action: move using arrow keys.
Condition: the cube enters the target area.
Visible consequence: the cube turns green and “Goal reached!” appears.

Dependencies: scene setup → working player movement → target detection → success feedback → final testing and evidence.

People: An Nguyen, working solo.

Tools: Unity, my C# editor, GitHub, and my notes.
Contributor: An Nguyen. The board is my own sketch.

Mission board copy · An Nguyen



My map connects character, goal, player action, condition, and visible consequence.

Character: a lost cube.
Goal: reach the marked destination.
Player action: move using arrow keys.
Condition: the cube enters the target area.
Visible consequence: the cube turns green and “Goal reached!” appears.

Dependencies: scene setup → working player movement → target detection → success feedback → final testing and evidence.

People: An Nguyen, working solo.
Tools: Unity, my C# editor, GitHub, and my notes.
Contributor: An Nguyen. The board is my own sketch.

Translate the mission into code.

My story-to-code map

Player cube → PlayerMover component → speed field and Speed property → Update reads arrow-key input and moves the cube → the player controls its position.

Target → GoalZone component → player reference and completed flag → OnTriggerEnter checks whether the player entered → CompleteGoal turns the cube green and displays “Goal reached!”

Instruction display → scene UI text → controls and goal message → shown when the scene starts → the player knows what to do.

What I am borrowing and why

I will use Unity’s built-in Transform, Collider, Rigidbody, Renderer, and UI capabilities. These provide movement, trigger detection, appearance, and visible instructions without adding an external package.

The project uses Unity 6.6 (6000.6.7f1). The built-in types come from UnityEngine; using UnityEngine makes those names available in the script. It does not install a package or attach a component.

I will adapt the movement and color ideas from my Action 11A scripts and credit that earlier work. I will record any additional code sources or AI assistance that I use.

Object lifetime and responsibility

The player, target, and UI will be saved scene objects created in the Unity Editor. I do not plan to spawn extra objects during play.

PlayerMover will handle input and movement. GoalZone will detect the player and trigger success once. I will disable the earlier automatic movement and boundary-stop components on the player in the new scene so they do not interfere.

I will test that success does not appear before reaching the target, appears once when the player enters, and resets when I stop and restart Play mode. If I create a runtime material for the color change, I will destroy that material in OnDestroy.

Make one moment playable.

What I expect to happen

WASD or arrow keys move Explorer. Pressing E outside Interaction Range 2 should do nothing. Pressing E within range should turn Beacon cyan and log the activation once. Increasing Speed from 3 to 5 should make movement faster without changing the interaction range.

What I built and where it lives

I built StoryScene11B in Week11B/EngineProject. The scene contains a Plane, Explorer with CharacterController and PlayerMover, and Beacon with StoryBeacon.

Scene: Assets/Scenes/StoryScene11B.unity
Scripts: Assets/Scripts/StoryScene11B/PlayerMover.cs and StoryBeacon.cs.

I adapted the supplied Action 11B example. Keyboard input moves Explorer, and pressing E within range calls Beacon.Activate(). Beacon turns cyan to represent restored communication.

A difficulty and my response

I initially needed help understanding how to open my previous scene and save a separate scene for 11B. I opened EngineTransfer11A and used Save As to create StoryScene11B.

I tested movement and interaction outside and inside the range. Repeated E presses did not activate the beacon again. After changing Speed from 3 to 5, Explorer moved faster while the interaction range stayed unchanged.

Test what you are claiming.

My test record

Movement: I expected WASD and arrow keys to move Explorer. Explorer moved as expected.

Outside range: I expected E to do nothing beyond Interaction Range 2. Beacon kept its original color.

Inside range: I expected E to activate Beacon. Beacon turned cyan.

Repeated activation: I expected further E presses to be ignored. The activation message appeared only once.

Evidence: outside-range.png, beacon-activated.png, and the Speed 5 Inspector screenshot.

Next action: organize the evidence and verify the saved scene and scripts before uploading.

My improvement experiment

Prediction: Increasing Speed from 3 to 5 would make Explorer move faster without changing the interaction range.

Change: I changed the PlayerMover Speed field from 3 to 5.

Observed result: Explorer moved faster. Pressing E outside the range still did nothing, while pressing E nearby activated Beacon.

Explanation: Speed controls movement, while interactionRange separately controls the distance required for activation.

Promised versus returned evidence

I planned a player-controlled character, a clear goal, and visible success feedback. I built Explorer movement and an interaction where the player approaches Beacon and presses E to restore communication.

The original plan used automatic target entry, a green cube, and a success label. I changed this to an E-key interaction within range and a cyan Beacon, with a Console message confirming activation.

My check-test-record routine helped me compare predictions with actual results. Next, I need to update my mission board to match the implemented interaction, document the Input System version and source, and upload the project and evidence to my personal GitHub.

Return with work you can explain.

My final access and file check

My local scene is StoryScene11B in Week11B/EngineProject. I tested movement, interaction outside and inside the range, repeated activation, and the Speed 3 to 5 change.

Final GitHub upload and remote access checks are still pending. I will upload the source, scene, dependency settings, report, board images, and test evidence, then open the uploaded files and links to verify them.

Working solo.

My next decision

I will improve the player's understanding by adding visible instructions for movement and pressing E near Beacon. I will also investigate showing the restored-communication message in the Game view instead of only in the Console.

Handoff

Commit and push your own project, notes, report, boards and tests. Verify file/image/link access. Include your team GitHub link if teamed. Submit one personal GitHub URL in your own Canvas record.

CS110 Action 11B in Canvas